

영화 추천 구현·코드 검증 가이드

장르 후보 선정 + 다중 메타데이터 + 리뷰 텍스트 + 보정 평점

검증일 2026-09-26 | 교육용 첫 구현 | 실제 CSV 기반 검증

결론

기존 PDF의 집계·벡터화·유사도 계산 아이디어는 재사용할 수 있습니다. 다만 원문 코드를 그대로 완성된 프로그램으로 사용하기는 어렵습니다. 결측치, 실제 추천 가능 영화 수, 자기 영화 제외, 입력 검증과 점수 결합 방식을 보완해야 합니다.

이번 문서가 제공하는 것

- ① 원본 코드의 재사용 판정 ② 실제 데이터 검사 결과 ③ 합의한 전체 추천 흐름 ④ 수정된 점수 설계와 코드 ⑤ 추천 영화의 리뷰를 평점순으로 보여주는 코드 ⑥ 독립 참조 구현 및 재검증 결과

검증 범위

실제 CSV의 집계와 NumPy/pandas 독립 참조 구현은 실행했습니다. scikit-learn과 FastAPI가 검증 환경에 없어 원문 sklearn/FastAPI 코드는 직접 실행하지 못했습니다. 따라서 원문 전체의 실행 성공이나 추천 품질 향상을 주장하지 않습니다.

함께 제공하는 recommendation_demo.py는 원문을 그대로 옮긴 코드가 아니라, 보완 설계를 검증하기 위한 실행 가능한 참조 코드입니다. 기존 학습 파일과 원본 CSV는 변경하지 않았습니다.

01. 합의한 추천 흐름

미리 준비

원본 데이터 확인 → 영화 식별·결측·중복 검사

영화별 집계 → 감독·장르·등급·상영구분·관객 수·스크린 수·평점·감성 비율·리뷰 수

영화별 리뷰 결합 → TF-IDF 생성 → 메타데이터 및 텍스트 유사도 저장

사용자 요청 시

1. 영화 제목 입력: 정확한 제목으로 조회하고, 동명 영화는 식별 정보로 구분
2. 같은 장르 후보 선정: 현재 데이터는 한 행에 단일 장르이며 멜로/로맨스는 하나의 장르
3. 메타데이터·리뷰 유사도에 감독 일치 가산점을 반영
4. 유사도 상위 최대 20개 후보 선정
5. 리뷰 수를 반영한 보정 평점과 유사도를 결합해 최종 정렬
6. 최대 5개 영화, 추천 점수·평점·리뷰 수·추천 이유 표시
7. 선택한 영화의 실제 리뷰를 높은 평점순 또는 낮은 평점순으로 조회

장르 미상은 임의로 다른 장르와 섞지 않고 추천 결과를 비웁니다. 후보가 부족하면 가능한 수만 반환합니다. 향후 다중 장르 데이터에서는 장르 집합의 교집합으로 필터를 확장합니다.

크롤링은 후속 확장입니다. 첫 버전은 현재 CSV로 동작하며 최신 리뷰 수집은 수행하지 않았습니다. Ridge 평점 예측은 별도 실험으로 유지합니다.

02. 실제 데이터 검사 결과

항목	확인 결과
원본	26,801행 · 22컬럼 · 영화명 216개
리뷰 결측	33행
원문 dropna 후 영화 수	214개: 문서의 216개와 다름
참조 구현 정리 후	26,768행 · 영화명 214개
장르별 프로필	드라마 163 / 코미디 34 / 멜로·로맨스 15 / 미상 2
추천 후보 자격	장르가 알려진 212개, 실제 후보 수는 장르마다 다름
프로필 결측	감독 6 / 상영구분 8 / 장르·등급·스크린·관객 수 각 2
메타데이터 충돌	검사한 7개 컬럼: 동일 영화명 내 서로 다른 유효값 0건
감성 라벨	0/1. 유효 행에서 pos_score > neg_score와 100% 일치

영화 식별 주의

리뷰 정리 후 제외된 제목: "그 시절, 우리가 좋아했던 소녀", "리향". 장르 미상: "그 시절 우리가 좋아했던 소녀", "초혼 다시 부르는 노래". 썸표 유무만 다른 제목이 발견됐지만 동일 작품인지 확인하지 않았으므로 자동 병합하지 않았습니다.

bert_label 평균은 기존 모델이 예측한 긍정 비율입니다. 사람의 직접 감성 정답이 아닙니다. 결측을 0(부정)으로 바꾸지 않습니다. review_id는 리뷰 ID이며 영화 ID로 사용하지 않습니다.

03. 원본 PDF 코드 재사용 판정

부분	판정과 수정 사항
groupby + first/mean	재사용 가능. 영화명 식별과 속성 충돌을 먼저 검사. 리뷰·평점·감성의 결측을 목적별로 처리.
StandardScaler	보완 필요. 원문 집계에도 숫자 결측이 남음. 결측 보완 없이 코사인 유사도 단계가 실패할 수 있음. 관객·스크린은 log1p 검토.
TF-IDF	재사용 가능. 프로필과 텍스트 행 순서를 명시적으로 정렬. 형태소/문자 n-gram 우월은 실험 필요. 명사만 추출하면 반드시 좋아지는 것은 아님.
0.4/0.6 벡터 결합	비중 해석 수정. 벡터 길이와 제공 가중치의 영향으로 유사도 40%/60%가 아님. 블록별 점수를 계산한 뒤 가중합.
정렬 후 첫 행 제외	수정 필요. 동점이면 자기 영화가 첫 행이 아닐 수 있음. 인덱스로 명시적 제외.
감독 가산점	재사용 가능. 감독 결측끼리는 일치로 취급하지 않음.
FastAPI 예시	골격으로 사용. app 생성, 모델 로딩, 입력 오류·미등록 영화 처리와 top_n 제한 추가 필요.

문서의 줄바꿈으로 문자열이나 슬라이스가 분리된 부분은 PDF 추출 텍스트를 그대로 붙여 넣지 말고 원래 파이썬 구문에 맞춰 복구해야 합니다.

04. 영화별 집계 코드

아래는 참조 코드의 핵심입니다. df는 원본 CSV입니다. 평점이 없는 리뷰도 텍스트 분석에는 보존하고, 평점 개수 v와 리뷰 수 n을 구분합니다.

CODE

```
d = df.copy()
d['review'] = d['review'].fillna('').str.strip()
d = d[d['review'].ne('')].drop_duplicates(
    ['MOVIE_NM', 'review', 'rating'])
g = d.groupby('MOVIE_NM', sort=True)
cols = ['DRCTR_NM', 'MOVIE_SDIV_NM', 'GRAD_NM',
        'GENRE_NM', 'OPN_DE', 'TOT_SCRN_CO',
        'VIEWNG_NMPR_CO']
p = g.agg(
    **{c: (c, 'first') for c in cols},
    R=('rating', 'mean'), v=('rating', 'count'),
    n=('review', 'size'), positive=('bert_label', 'mean'))
p['text'] = g['review'].apply(
    lambda s: ' '.join(s.drop_duplicates()))
```

first는 현재 검사에서 유효값 충돌이 없다는 조건으로 사용합니다. 다른 데이터에서는 가장 최신 값인지, 잘못된 묶인 동명 영화인지 먼저 확인해야 합니다.

v는 평점이 존재하는 보존 행 수이며 고유 사용자 수가 아닙니다. 같은 문구에 다른 평점이 있으면 평점 행은 보존하되 텍스트는 한 번만 합칩니다. 실제 평점 범위는 0.5~5.0입니다.

기존 평점 예측 데이터와 차이

평점 예측에서는 review/rating이 모두 필요해 ready 파일이 26,321행이었습니다. 추천 텍스트에는 평점 없는 리뷰도 사용할 수 있고 영화별 중복 기준도 다르므로, 이번 집계의 행 수가 달라지는 것이 정상입니다.

05. 특징·유사도와 최종 점수

참조 구현은 원문의 OneHotEncoder/StandardScaler 대안을 사용했습니다. 범주형은 일치 비율, 수치형은 정규화된 거리로 메타데이터 유사도를 만듭니다. 원문과 동일한 점수가 나오는 구현은 아닙니다.

범주형: 상영구분·등급의 일치 비율. 결측은 미상으로 표시합니다. 미상끼리 일치하는 효과는 초기 구현의 한계이며, 운영 시 해당 속성을 비교에서 제외하는 정책도 실험합니다.

수치형: 관객·스크린 수에 $\log_{1p}$ 적용, 결측은 중앙값 보완. 평점·공정 비율과 함께 각 열을 0~1로 변환한 뒤 "1 - 평균 절대 차이"를 계산합니다.

CODE

```
meta = 0.5 * cat_sim + 0.5 * num_sim
# 문자 2~3그램 TF-IDF, min_df=2, 최대 5,000개
# 로그 TF, smooth IDF, 행별 L2 정규화
text_sim = x @ x.T
# director_match는 감독이 알려져 있고 같은 경우만 1
S = clip(0.4 * meta + 0.6 * text_sim
        + 0.05 * director_match, 0, 1)
```

리뷰 수를 반영한 평점

CODE

```
B = (v * R + m * C) / (v + m)
Q = clip((B - 0.5) / 4.5, 0, 1)
final_score = 0.7 * S + 0.3 * Q
```

R: 영화 평균 평점 / v: 유효 평점 수 / C: 전체 유효 평점 평균 3.0616 / m: 보정 강도 20. 평점이 없는 영화는 R을 C로 보완합니다.

가중치·m·후보 20개는 시작용 설정이며 최적값이 아닙니다. 평점이 S와 Q에 중복 반영되므로, 메타데이터에서 평점을 뺀 설정과 비교해야 합니다. C와 m은 계산한 보정 기준이지 통계적 신뢰구간이 아닙니다.

06. 추천 후보 필터와 정렬 코드

p, meta, text_sim은 앞 단계의 결과입니다. 전체 실행 코드는 동봉한 recommendation_demo.py에 있습니다.

CODE

```
def recommend(name,top_n=5):
    if name not in p.index: raise ValueError('영화명을 찾을 수 없습니다.')
    if not 1<=top_n<=20: raise ValueError('top_n은 1~20이어야 합니다.')
    i=p.index.get_loc(name); genre=p.iloc[i]['GENRE_NM']
    if pd.isna(genre): return pd.DataFrame()
    director=p.iloc[i]['DRCTR_NM']
    same = np.zeros(len(p), bool)
    if pd.notna(director):
        same = p.DRCTR_NM.eq(director).fillna(False).to_numpy()
    s=np.clip(0.4*meta[i]+0.6*text_sim[i]+0.05*same,0,1)
    out=p.copy(); out['similarity']=s
    keep = out.GENRE_NM.eq(genre).fillna(False)
    out = out.loc[keep & (out.index != name)]
    out=out.sort_values('similarity',ascending=False,kind='stable').head(20)
    out['score']=0.7*out.similarity+0.3*out.Q
    return out.sort_values('score',ascending=False,kind='stable').head(top_n)
```

먼저 유사도 상위 20개를 고르고 그 안에서 최종 점수로 정렬합니다. 순수 평점순 정렬이 아니므로, 평점이 조금 낮더라도 더 비슷한 영화가 위에 올 수 있습니다.

장르가 미상인 경우 빈 결과, 없는 제목은 ValueError입니다. API에서는 이를 사용자에게 읽기 쉬운 응답으로 변환해야 합니다.

07. 선택 영화의 리뷰 정렬·표시

추천 영화 목록의 순위와, 한 영화 안에서 보여줄 리뷰 순서는 별도입니다. 아래 함수는 실제 평점이 있는 리뷰를 높은 점수순 또는 낮은 점수순으로 보여줍니다.

CODE

```
def show_reviews(movie_name, top_n=5, order='high'):
    if movie_name not in p.index:
        raise ValueError('영화명을 찾을 수 없습니다.')
    if order not in ('high', 'low') or not 1 <= top_n <= 20:
        raise ValueError('정렬 방식과 개수를 확인하세요.')
    rows = d.loc[d.MOVIE_NM.eq(movie_name),
                 ['review', 'rating']].copy()
    rows = rows.dropna(subset=['rating'])
    return rows.sort_values(
        'rating', ascending=(order == 'low'),
        kind='stable').head(top_n)

print(recommend("승부")[['R', 'V', 'similarity', 'score']])
print(show_reviews('서울의 봄', order='high'))
print(show_reviews('서울의 봄', order='low'))
```

평점 동점은 원본 순서를 유지합니다. 이것은 최신순·도움순·리뷰 품질순이 아닙니다. 평점이 없거나 빈 리뷰는 이 표시 목록에서 제외됩니다.

추천 화면에서 높은 평점 리뷰만 보여주면 편향된 인상을 줄 수 있습니다. 높은/낮은 평점 보기 전환과 전체 평균·평점 수를 함께 제공하세요. 원문 리뷰는 저장소에 재배포하지 않았습니다.

이 기능은 기존 bert_label이나 Ridge 예측값을 실제 사용자 평점처럼 표시하지 않습니다. “리뷰를 이용한 영화 정렬”은 앞 단계의 text_sim에 이미 반영되어 있습니다.

08. 실행 결과와 자체 검증

예시 입력: 승부. 아래는 2026-09-26 로컬 CSV에서 독립 참조 구현을 실행한 결과이며, 추천 품질을 보증하는 정답 목록은 아닙니다.

추천 영화	평균 평점	평점 수	유사도	최종 점수
서울의 봄	3.878	193	0.827	0.799
빅토리	3.282	202	0.808	0.750
행복의 나라	2.887	194	0.821	0.735
비상선언	2.711	178	0.817	0.721
1승	2.715	191	0.814	0.719

자동 검사 통과

214개 영화 입력에 대해 자기 영화 제외, 중복 추천 없음, 같은 장르 유지, 최대 5개, 유한 점수, 최종 점수 내림차순을 검사했습니다. 장르 미상 2개는 빈 결과를 반환합니다.

없는 영화명 및 top_n=0/21 오류 처리를 검사했습니다. 텍스트 유사도 행렬의 대칭·대각선 1, 텍스트와 프로필의 행 정렬도 확인했습니다.

214개 영화 × 높은/낮은 평점 정렬 = 428개 경우를 확인했습니다. 각 리뷰 목록의 순서, 최대 개수, 빈 리뷰 제외 조건이 통과했습니다.

검증의 경계

실행 환경: Python 3.12, NumPy 2.3.5, pandas 3.0.1. scikit-learn 설치를 시도했지만 사용 가능한 배포본을 확보하지 못했습니다. 원문 sklearn/FastAPI의 통합 실행, 새 PC 설치, 크롤링, 사용자 선호 평가는 미검증입니다.

실행 검증 통과는 “추천이 사용자 취향에 맞는다”는 뜻이 아닙니다. 장르+보정 평점 기준과 비교하고, 사람이 추천 사례를 평가한 뒤 가중치를 조절해야 합니다.

09. 실행 방법과 다음 학습 단계

recommendation_demo.py를 새 실습 폴더에 저장하고 원본 CSV를 같은 폴더에 두세요. 기존 모델 코드를 덮어쓰지 않습니다.

CODE

```
python -m pip install numpy pandas
python recommendation_demo.py

# 원본 CSV가 다른 폴더에 있는 경우
python recommendation_demo.py "D:/JungPra/pythonpra/NSMC/Koreanmovie_review.csv"
```

실행하면 실제 데이터 검사, 유사도 및 추천 계산, 리뷰 정렬 검사를 수행하고 verification.json을 스크립트 폴더에 저장합니다. 기존 검증 JSON은 덮어씁니다. 원본 CSV는 읽기만 합니다. 검사 실패 시 assert/ValueError로 중단됩니다.

학습 순서

1. 영화별 집계와 결측치 확인
2. TF-IDF 특징 확인과 영화 리뷰 결합
3. 각 유사도 및 보정 평점의 의미 설명
4. 후보 필터와 점수 정렬
5. 추천 이유와 리뷰 표시
6. 기존 모델 비교와 가중치 실험
7. FastAPI와 크롤링은 후속 단계

자료와 재현 기록

설계 참고: 콘텐츠기반_영화추천_구현가이드.pdf, 4쪽. 원문 아이디어를 바탕으로 코드 검토와 수정 설계를 수행했습니다.

실측 데이터: D:/JungPra/pythonpra/NSMC/Koreanmovie_review.csv. 데이터 출처: Kaggle의 Korean Movie Review Data 30k(BERT). 외부 최신 정보나 새 리뷰는 수집하지 않았습니다.

입력 SHA-256:

```
10e986c9ceda77e0606f6555d1c0abf3
f5f55f35a5278f78b9f5d430d882e9f3
```

추천에 사용할 영화 ID 부재, 제목 변형, 단일 장르와 결측 속성, 표본 편향, 플랫폼 평점 척도 미확인은 후속 과제로 남깁니다. 새 데이터가 추가되면 집계·특징·유사도를 일관되게 다시 생성하세요.

10. 평가 수에 따른 평점 보정

베이지안 평균 방식의 보정입니다. 평가가 적으면 전체 평균을 더 반영하고, 많으면 해당 영화의 평균을 더 반영합니다. 영화 평균과 전체 평균을 평가 수에 따라 가중합하는 수축 추정 방식입니다.

CODE

$$B = (v * R + m * C) / (v + m) \\ = (v / (v + m)) * R + (m / (v + m)) * C$$

기호	의미와 이번 구현
R / v	영화 평균 평점 / 실제 평점이 있는 보존 행 수
C	정리 후 전체 유효 평점의 평균: 약 3.0616점
m	보정 강도: 초기 실험값 20. 최적값으로 검증하지 않음.
B	추천 순위 계산에 사용하는 보정 평점

계산 예시: 설명을 위해 $C=3$, $m=20$ 으로 가정

영화	평점 수 v	평균 R	보정 B
A	2	5.0	3.18
B	200	4.5	4.36
C	200	5.0	4.82

$m=20$ 은 전체 평균 점수의 가상 평가 20개를 섞는 것과 같은 계산 효과입니다. 20개 미만인 영화를 제외한다는 뜻은 아닙니다. $v=m$ 이면 영화 평균과 전체 평균을 절반씩 반영합니다.

평가가 적다고 항상 점수가 내려가지는 않습니다. 전체 평균보다 낮은 평점은 위로 보정됩니다. 고유 평가자 수, 평점 분산, 허위 리뷰나 수집 편향을 직접 반영하는 공식은 아닙니다.

공식 출처와 적용 범위

[IMDb 공식 Ratings FAQ - Top 250 순위 계산 공식](#) (확인: 2026-09-26). 위와 같은 가중합 형태를 설명합니다. IMDb의 m 은 차트 진입 최소 평가 수와 연결되지만, 여기서는 보정 강도로만 사용했습니다.

IMDb 일반 영화 페이지의 상세 평점 산식은 비공개입니다. 이 구현은 IMDb 전체 시스템을 재현한 것이 아닙니다. $m=5, 20, 50$ 에 따른 순위 변화를 비교하는 실험이 다음 과제입니다.